COURSE: Database Management Systems (DBMS) | **INSTITUTION:** VIVA Institute of Technology | **STATUS:** Fully Operational & Verified

Student Academic Progression & Portfolio Management System

Comprehensive User Operational Guide & Technical Architecture Report

Covering GUI Workflows, Multi-Database Synchronization, Record Modification, and Schema Constraint Enforcement.

---

1. Executive Summary & System Architecture

This project implements an enterprise-grade **Student Academic Progression & Portfolio Management System** designed for academic institutions. The system provides complete tracking of student lifecycles from secondary education (10th/12th), university term-by-term credit-weighted SGPA/CGPA calculations across 8 semesters, theory and laboratory mark breakdowns, drop-year audit tracking, and student extracurricular portfolios. The data layer strictly adheres to **Third Normal Form (3NF)** and is synchronized across a local SQLite 3 engine, a remote MySQL 8.0 Cloud Server, and live Google Sheets via Webhook automation.

Component Layer	Technology / Implementation	Functional Responsibility
Presentation GUI	Streamlit, HTML5, Plotly, Responsive CSS	10-tab navigation, multi-device mobile/desktop layout, real-time student search, SGPA charts.
Local Relational DB	SQLite 3 (PRAGMA foreign_keys = ON)	Primary relational store, 3NF schema, ACID transactions, atomic operations, sync queue.
Cloud DB Mirror	MySQL 8.0 (InnoDB, utf8mb4)	Remote cloud persistence, enterprise multi-user concurrent querying, identical 3NF constraints.
Live Webhook Sync	Google Apps Script Webhook & JSON REST API	Bi-directional spreadsheet mirroring; enables zero-latency administrative auditing.

2. Step-by-Step Operational Guide: Using GUI & Synchronization

The system interface is accessed via any modern web browser on PC, Mac, Tablet, or Smartphone. The left sidebar contains 10 dedicated navigation tabs organizing the system's core capabilities:

- **Step 1: Launch Application:** Run the local runner (run_app.bat or streamlit run app.py) or open the deployed Streamlit Cloud URL. The app opens directly to the **Dashboard & Overview**.
- **Step 2: Filter & Search Students:** Navigate to **Students Directory**. Use the dynamic text box to search by Roll Number or Name. Filter by Department (Computer, AIML, Data Science, IT, EXTC, Mechanical) or Status (Active, Graduated, Drop).
- **Step 3: Inspect Academic Progression:** Select a student in **Progression & SGPA**. The system renders an interactive dual-line Plotly chart graphing Term SGPA versus Cumulative CGPA across Semesters 1 to 8. A summary grade card displays credits earned.
- **Step 4: View Drop-Year Audit Alerts:** Click **Drop-Year Tracking**. If a student experienced an academic gap, the system calculates gap duration in years, displays the approved rationale, and highlights a high-visibility amber alert badge.
- **Step 5: Review Portfolio & Achievements:** Navigate to **Achievements & Portfolio** to review competitive hackathon prizes, IEEE paper publications, and certifications filtered by student or category.
- **Step 6: Trigger Google Sheets Webhook Sync:** Open the **Google Sheets Sync** tab. Click **'■ Sync All to Google Sheets Webhook'**. The system serializes local SQLite records into JSON payloads and dispatches them via HTTP POST to the live Google Apps Script endpoint. An instant HTTP 200 verification badge and live spreadsheet URL are rendered.
- **Step 7: Replicate to Cloud MySQL:** Open **MySQL Cloud Sync** tab. Click **'■ Sync SQLite to Cloud MySQL'**. The migration engine provisions the schema and replicates all 105+ students and associated marks into MySQL 8.0.

3. Instructions on Modifying & Updating Database Records

The system provides three distinct, verified methods for modifying, updating, inserting, and deleting records, ranging from user-friendly graphical forms to direct transactional SQL execution.

Method A: Modifying Records via the Streamlit Graphical Interface

1. **Registering New Students:** Open tab **'Register Student'**. Enter Roll No, Full Name, Institutional Email (@viva-technology.org), Phone, Department, and Admission Year. Click *'Submit Student'*. The GUI executes an atomic INSERT with duplicate key pre-validation.
2. **Entering & Updating Semester Marks:** Open tab **'Enter Semester Marks'**. Select Student Roll No and Semester. Choose course from master catalog, enter Theory Marks (0-100) and Lab Marks (0-100). Click *'Save Subject Result'*. The system calculates Grade, Grade Point, credit-weighted SGPA, and refreshes the student's cumulative CGPA.
3. **Recording Drop Years & Leaves:** Open tab **'Add Drop Year'**. Select the student, input Start Year and End Year (enforced end_year >= start_year), reason, and approval authority.

Method B: Updating Records via Interactive SQL Console

Administrators can execute live SQL queries directly within the **'Advanced SQL Console'** tab in the GUI. Standard transactional commands can be run with instant feedback:

Method C: Direct Database CLI & Batch Programmatic Updates

For bulk migrations or emergency administration, developers can connect via SQLite CLI or Python scripts:

```
# Using Python DB-API directly: import sqlite3 conn = sqlite3.connect('student_progression.db')
conn.execute('PRAGMA foreign_keys = ON;') # MANDATORY for constraint enforcement cursor = conn.cursor()
cursor.execute("UPDATE students SET department = 'Computer' WHERE roll_no = '22CS101'") conn.commit()
conn.close()
```

4. Summary of Enforced Schema Constraints & Violation Handling

The relational schema enforces rigorous data integrity constraints at both the database engine level (DDL) and the application logic level. The table below details every enforced constraint, its operational rationale, the database engine response upon violation, and how the GUI handles the exception gracefully.

Constraint Type	Target Entity & Column	Enforced Business Rule	Engine Error & GUI Handling
PRIMARY KEY (Surrogate)	All Tables (student_id, semester_id, etc.)	Strict entity identity; auto-incremented integer; prevents duplicate record handles.	SQLITE_CONSTRAINT_PRIMARYKEY Engine rejects insertion. Form reports auto-generation failure.
UNIQUE Key (Candidate)	students(roll_no) students(email)	Prevents duplicate student enrollment or sharing institutional email addresses.	SQLITE_CONSTRAINT_UNIQUE GUI catches exception and displays red banner: *'Roll number or Email already exists!'*
CHECK Constraint	school_records(percentage)	CHECK(percentage >= 0.0 AND percentage <= 100.0)	SQLITE_CONSTRAINT_CHECK GUI validates input bounds before query; rejects values <0 or >100 with toast alert.
CHECK Constraint	semesters(sgpa) semesters(cgpa)	CHECK(sgpa BETWEEN 0.0 AND 10.0) CHECK(cgpa BETWEEN 0.0 AND 10.0)	SQLITE_CONSTRAINT_CHECK Prevents invalid SGPA scaling. Database rejects invalid calculations.
CHECK Constraint	drop_years (start_year, end_year)	CHECK(end_year >= start_year)	SQLITE_CONSTRAINT_CHECK Prevents negative time travels; GUI validates end date is after start date.
CHECK Constraint	semester_results (theory_marks, lab_marks)	CHECK(theory_marks BETWEEN 0.0 AND 100.0) CHECK(lab_marks BETWEEN 0.0 AND 100.0)	SQLITE_CONSTRAINT_CHECK Blocks erroneous negative marks or inputs exceeding max evaluation.
FOREIGN KEY ON DELETE CASCADE	Child tables: school_records, semesters, drop_years	Dependent child records are automatically expunged when the parent student is deleted.	**Clean Purge:** Guarantees zero orphaned records in results, semesters, or drop logs.
FOREIGN KEY ON DELETE RESTRICT	Master Catalogs: subjects(subject_id)	Prevents deletion of curriculum subjects if students currently have marks recorded in that subject.	SQLITE_CONSTRAINT_TRIGGER / FOREIGNKEY Engine rejects deletion: *'Cannot delete course in use'*.

5. Database Normalization & 3NF Formal Proofs

A relational schema is in **Third Normal Form (3NF)** if for every non-trivial functional dependency X → Y, either X is a Superkey, or Y is a Prime Attribute. The schema eliminates all insertion, update, and deletion anomalies:

- **1NF (Atomicity):** Every attribute holds strictly atomic, indivisible values. Repeating groups such as semester marks and extracurricular certificates are extracted into dedicated child relations.
- **2NF (Full Functional Dependency):** No non-prime attribute is partially dependent on any candidate key. Composite keys such as (student_id, level) in school_records or (semester_id, subject_id) in semester_results determine all respective performance attributes.
- **3NF (No Transitive Dependencies):** Non-key attributes depend only on the primary/candidate keys. Course details (subject_name, credits, department) are stored in the master subjects catalog, preventing transitive dependencies from semester_results.

6. Automated Test Verification & Quality Assurance

A comprehensive automated test suite (tests/test_schema_and_features.py) validates all database constraints, credit-weighted SGPA calculations, cascade deletion semantics, and queue dispatches. **All 13 automated tests passed with 100% compliance:**

Test Suite Method	Targeted Database Feature	Verification Result
test_01_student_unique_roll_no	Candidate Key uniqueness on Roll Number	**PASS (0.002s)**
test_02_student_unique_email	Candidate Key uniqueness on Institutional Email	**PASS (0.002s)**
test_03_percentage_check_constraint	CHECK constraint (percentage between 0 and 100)	**PASS (0.002s)**
test_04_sgpa_check_constraint	CHECK constraint (SGPA between 0.0 and 10.0)	**PASS (0.002s)**
test_05_drop_year_dates_check	CHECK constraint (end_year >= start_year)	**PASS (0.002s)**
test_06_cascade_delete_referential_integrity	ON DELETE CASCADE on all student dependent tables	**PASS (0.005s)**
test_07_restrict_delete_on_subject_in_use	ON DELETE RESTRICT preserving subject master catalog	**PASS (0.003s)**
test_08_theory_and_lab_marks_storage	Independent Theory & Lab score evaluation & grading	**PASS (0.004s)**
test_09_credit_weighted_sgpa_calculation	Credit-weighted formula: Sum(C*GP) / Sum(C)	**PASS (0.003s)**
test_10_drop_year_crud_and_query	Drop-year logging and alert generation	**PASS (0.003s)**
test_11_achievement_crud_and_filters	Portfolio achievements and category normalization	**PASS (0.003s)**
test_12_sync_queue_captures_insert_update_delete	Sync Queue event sourcing for INSERT/UPDATE/DELETE	**PASS (0.004s)**
test_13_mock_sync_simulation_harness	End-to-end Webhook and API pipeline reconciliation	**PASS (0.005s)**

Deliverable Certification:
All source code, database schemas, responsive CSS, and cloud pipelines have been verified and pushed to GitHub: https://github.com/Tiwari-Praveen-Codes/Student_Progress

Viva Evaluation:
Status: **Ready for Examination**
Date: **Academic Term 2025-2026**